

Experiment 1

Module Name: Python BDD Restful Automations

Experiment Name: Python BDD and Behave Framework Setup

Problem statement: Traditional automation scripts can be difficult for non-technical stakeholders (business analysts, product owners) to read and understand. There is a need to bridge the communication gap between technical implementation and business requirements using a human-readable language.

Objective: Setup Python BDD framework and debug and run automation framework for end to end scenarios.

Tools, software, and concepts used: Python 3, Selenium WebDriver, Behave BDD framework, Gherkin syntax (Feature, Scenario, Given, When, Then), PyCharm IDE.

Source code or implementation steps:

1. Create a folder inside that folder create two folders 'features' and 'steps'
2. Inside that 'features' folder create a file 'login.feature' and write this
Feature: SauceDemo Login Functionality

Scenario: Successful login with valid credentials
Given the user navigates to the SauceDemo login page
When the user enters a valid username and password
And the user clicks the login button
Then the user should be redirected to the inventory dashboard

3. Inside that 'steps' folder create 'login_steps.py' and write this

```
from behave import given, when, then

from selenium import webdriver
from selenium.webdriver.common.by import By

@given('the user navigates to the SauceDemo login page')
def step_impl(context):
    context.driver = webdriver.Chrome()
    context.driver.maximize_window()
    context.driver.get("https://www.saucedemo.com/")

@when('the user enters a valid username and password')
def step_impl(context):
    context.driver.find_element(By.ID, "user-name").send_keys("standard_user")
    context.driver.find_element(By.ID, "password").send_keys("secret_sauce")

@when('the user clicks the login button')
def step_impl(context):
    context.driver.find_element(By.ID, "login-button").click()

@then('the user should be redirected to the inventory dashboard')
def step_impl(context):
```

```
assert "inventory.html" in context.driver.current_url
context.driver.quit()
```

4. Now in your created folder go to cmd and write this

```
>>> behave
```

Output with clearly labelled screenshots:

```
D:\My Desktop\Selenium>behave
USING RUNNER: behave.runner:Runner
Feature: SauceDemo Login Functionality # features/login.feature:1

    Then the user should be redirected to the inventory dashboard # steps/login_steps.py:20 2.349s
    Then the user should be redirected to the inventory dashboard # steps/login_steps.py:20
1 feature passed, 0 failed, 0 skipped
1 scenario passed, 0 failed, 0 skipped
4 steps passed, 0 failed, 0 skipped
Took 0min 9.290s
```

Screenshot 1: Terminal output showing successful execution of the 'behave' command

Console Output:

USING RUNNER: behave.runner:Runner

Feature: SauceDemo Login Functionality # features/login.feature:1

Then the user should be redirected to the inventory dashboard # steps/login_steps.py:20
2.349s

Then the user should be redirected to the inventory dashboard # steps/login_steps.py:20

1 feature passed, 0 failed, 0 skipped

1 scenario passed, 0 failed, 0 skipped

4 steps passed, 0 failed, 0 skipped

Took 0min 9.290s

Brief result: The Behave framework was successfully initialized, and the Gherkin feature file correctly executed the underlying Selenium Python steps.

Observation: The context object in Behave acts as a global dictionary, allowing us to seamlessly pass the webdriver instance between different step definition functions.

Conclusion: Implementing BDD with Behave allows for "living documentation." The tests are written in plain English, making it easy for non-developers to verify exactly what the automation script is testing.